ML PROJECT · PARTIAL IMPLEMENTATION REVIEW

Temporal Feature Fusion

Regime-Aware Financial Trend Classification

Using 1D-CNN + Self-Attention

Kanishk Srivastava · Raghav Sharma · Sayani Chanda

Domain: Financial Machine Learning | Time-Series Classification

01 Architecture

1D-CNN + Attention
Blueprint

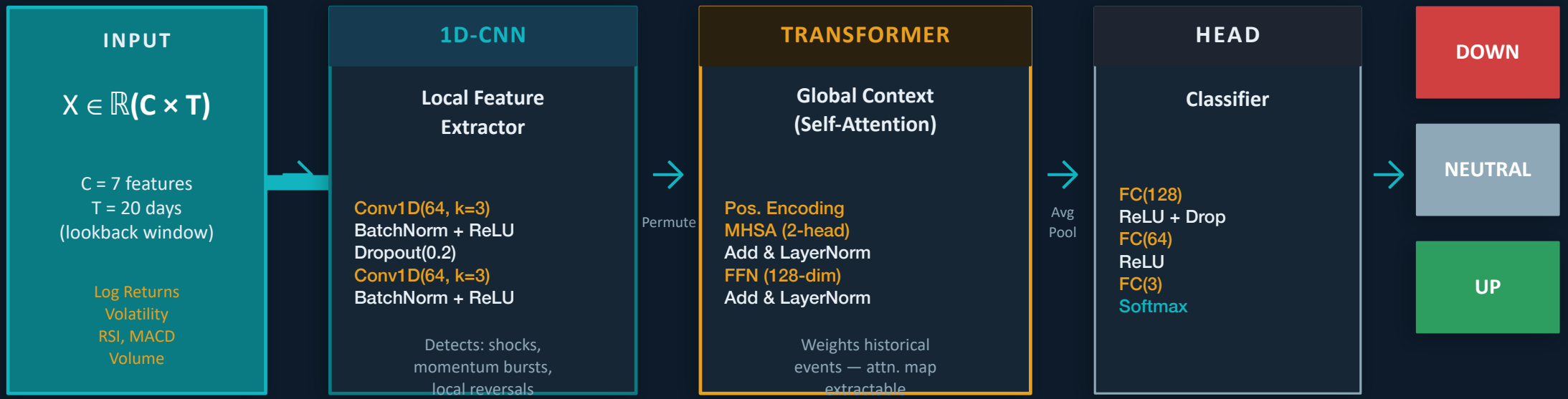
02 Dataset

Curated, Stationary
Spec Sheet

03 Partial Implementation

Regime-Aware
Labeler Code

01 · ARCHITECTURE — THE TEMPORAL FUSION PIPELINE



KEY INSIGHT vs. CNNPred: CNNPred applied 2D convolutions (3×3 filters) treating finance data as a spatial image — forcing false relationships between RSI and Volume. Our 1D-CNN slides filters along the TIME axis only. No spatial assumption. No incoherence.

Why Attention?

CNN cannot connect a volatility spike 15 days ago to today's prediction without massive depth. Attention does it in one step — and the weight map is extractable for interpretability.

Why lightweight? (1-2 heads, 1 layer)

We have T=20 time steps. A full 12-layer transformer would overfit immediately. One encoder layer is enough to prove the concept on a 20-day lookback.

02 · DATASET — THE CURATED SPEC SHEET

Source	Yahoo Finance API (yfinance)
Asset	S&P 500 Index (^GSPC)
Date Range	Jan 1, 2013 → Dec 31, 2023 (11 years)
Resolution	Daily (Δt = 1 trading day)
Total Rows	~2,770 trading days after warm-up removal
Stationarity	Raw prices NEVER used — all features derived from returns

#	Feature	Why
1–4	Log-Returns (OHLC)	Stationarity guaranteed
5	Rolling Vol σ (20d)	Noise floor for barrier ε
6	RSI-14 (norm. 0–1)	Momentum state
7	MACD (signal diff)	Trend cross-over signal

NOTE ON STATIONARITY

All features are log-returns or normalized derivatives computed exclusively from PAST data. Raw price levels are never fed to the model. Rolling normalization (z-score) is fitted ONLY on each training fold, then applied to the test fold — preventing normalization leakage across the time boundary.

03 · PARTIAL IMPLEMENTATION — REGIME-AWARE LABELER

data_ingestion.py — LabelGenerator.generate()

```
def generate_labels(df, λ=1.0, k=5, vol_window=30):
    # —— Step 1: FUTURE LOG-RETURN (label only, never a feature)
    close = df['Close']
    R_t = log(close.shift(-k) / close)        # look FORWARD

    # —— Step 2: DYNAMIC BARRIER (past data only — zero leakage)
    log_ret = log(close / close.shift(1))
    σ_t     = log_ret.rolling(vol_window).std()
    ε_t     = λ × σ_t                  # ε adapts to regime

    # —— Step 3: ASSIGN 3-CLASS LABELS
    labels = pd.Series(np.nan, index=df.index)
    labels[R_t  >  ε_t]  = 2    # Up
    labels[R_t  < -ε_t]  = 0    # Down
    neutral = (R_t >= -ε_t) & (R_t <= ε_t)
    labels[neutral]      = 1    # Neutral

    return
labels.reindex(feature_index)
```

① Stationarity Lock

Raw prices never used. Log-returns (log Close_t / Close_{t-1}) enforce stationarity by removing price-level drift. The model cannot learn to predict levels.

② Zero Leakage — Barrier

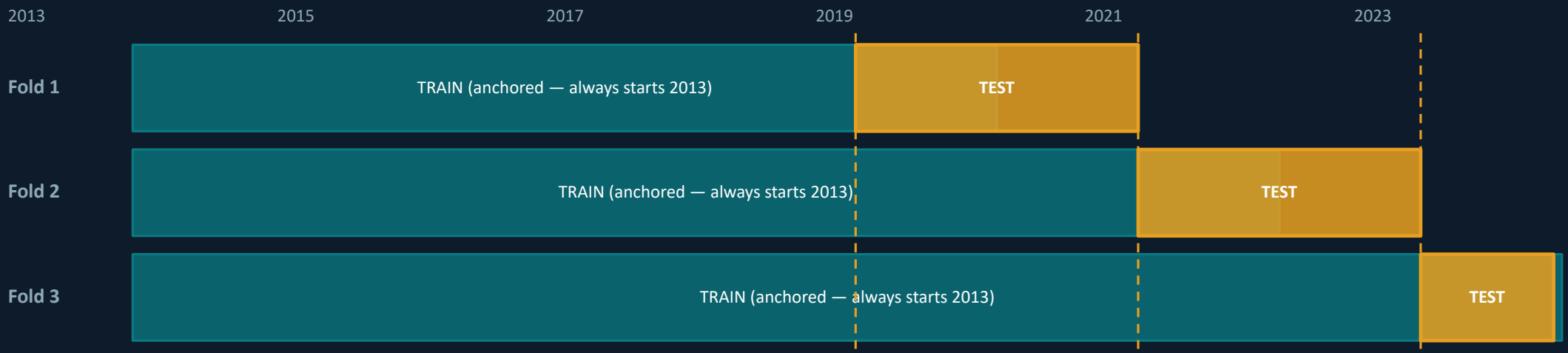
Rolling std uses only PAST 30 days of log-returns (shift forward, not backward). ε_t adapts to current market stress — it is SMALLER in calm markets, LARGER in volatile ones.

③ λ Grid Search Defense

λ is NOT guessed. We test {0.5, 1.0, 1.5} on training data only and select the value that keeps Neutral class between 35%–50%. This is the first question any examiner will ask.

04 · VALIDATION — ANCHORED WALK-FORWARD (ZERO LEAKAGE)

ANCHORED (chosen) — training window grows, never discards history



NORMALIZATION RULE (enforced in code):

Z-score mean and std are fitted ONLY on each training fold, then applied to the corresponding test fold. No test statistics touch the normalization step. This is implemented in FoldNormalizer in walk_forward.py.

Rolling Walk-Forward (REJECTED)

Discards old data. With only ~2,770 daily samples, early folds would have dangerously small training sets. Also harder to defend — 'why did you throw away 5 years of data?'

Anchored Walk-Forward (CHOSEN)

Training set only gets stronger with each fold. Defensible: 'We use all available history to make each forecast.' More stable results. Less variance across folds.

STATUS & NEXT STEPS

✓ COMPLETED

- ✓ Architecture designed — 1D-CNN + Transformer encoder
- ✓ Dataset curated — S&P 500, 11 years, 7 stationary features
- ✓ Regime-Aware Labeler implemented (LabelGenerator class)
- ✓ Lambda grid search procedure coded (0.5 / 1.0 / 1.5)
- ✓ Anchored walk-forward splitter implemented (3 folds)
- ✓ Per-fold normalization with zero leakage (FoldNormalizer)
- ✓ All 5 model skeletons written (Proposed, CNN, LSTM, XGB, Dummy)

→ IN PROGRESS

→ **Full model training loop**	*PyTorch + early stopping*
→ **Baseline training**	*XGBoost, LSTM, Dummy*
→ **Evaluation module**	*Macro F1, MCC, confusion matrix*
→ **Regime analysis**	*Low-vol vs High-vol F1 split*
→ **Attention visualization**	*Heatmap over 20-day lookback*
→ **Results table**	*Per-fold + aggregated comparison*
→ **Final write-up**	*Methodology + honest framing*

The theory phase is complete. The data integrity engine is built. The hardest mathematical problem — correct labeling without leakage — is solved.